Discipline : Enseignement de spécialité NSI

Contenus Graphes : structures relationnelles. Sommets, arcs, arêtes, graphes orientés ou non orientés.

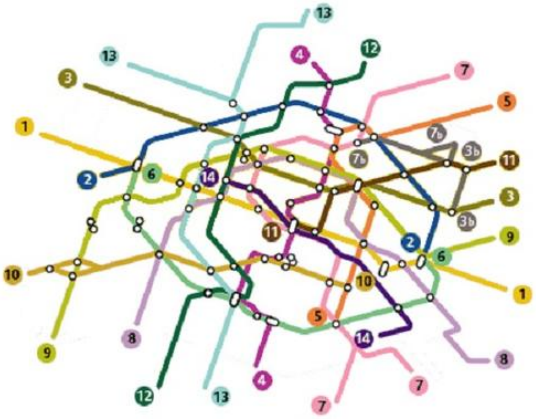
Algorithmes sur les graphes.

Capacités attendues : Modéliser des situations sous forme de graphes. Écrire les implémentations correspondantes d'un graphe : matrice d'adjacence, liste de successeurs/de prédécesseurs. Passer d'une représentation à une autre.

Parcourir un graphe en profondeur d'abord, en largeur d'abord. Repérer la présence d'un cycle dans un graphe. Chercher un chemin dans un graphe.

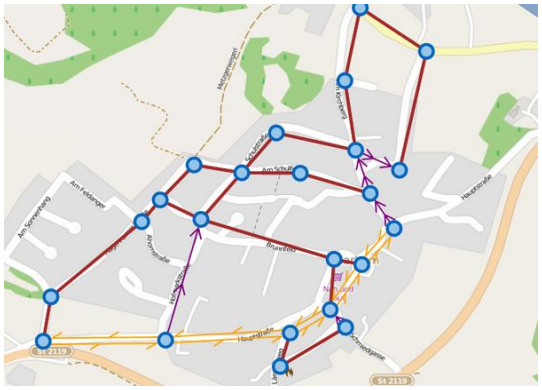
Pré-requis : Les arbres

A Les Graphes



Dans un graphe, les sommets peuvent être reliés à autant de nœuds que nécessaire et il peut comporter des boucles à la différence des arbres. En partant de certains nœuds, on est capable de revenir sur le nœud de départ. Les arêtes peuvent avoir une valeur numérique, un poids, on parle alors d'arbre pondéré. Selon la nature du graphe, les arêtes peuvent avoir un sens et des poids différents selon le sens, on parlera alors d'arcs. Le graphe est défini par ses sommets et ses arêtes.

Si les arêtes qui relient les sommets n'ont pas de sens, on dit que le graphe est non orienté. On peut indifféremment aller du sommet A vers B que de B vers A.



Si les arêtes ont un sens, on emploie le mot arc. On parlera alors de graphe orienté. On pourra par exemple pouvoir aller de A vers B mais pas l'inverse. Dans le cas d'un graphe orienté, on peut aussi avoir des poids différents selon l'arc. Par exemple 10 pour aller de A vers B, mais 8 pour aller de B vers A. Imaginez deux villes qui dans un sens sont reliées par l'autoroute mais pas dans l'autre. Le temps mis dans un sens ne sera pas celui mis dans l'autre.

Un chemin est une suite de sommets tel qu'il existe une arête reliant chacun de ses sommets.

Un graphe est connexe s'il existe toujours un chemin pour relier deux sommets.

Un cycle est une séquence d'au moins trois sommets où le sommet de départ=sommet d'arrivé et tous les autres sommets sont différents. On revient au point de départ.

On peut maintenant redéfinir nos arbres avec le vocabulaire des graphes. Un arbre est un graphe connexe, sans cycle, dont un sommet est particulier: la racine.

Un graphe est dit complet si l'on peut aller directement d'un sommet à un autre.

Les graphes se retrouvent dans de très nombreux domaines. Organiser une tournée de distribution de lettres, cheminement d'un colis, trajet d'une ville à un autre (train, avion, voiture, optimiser sur le temps, sur le prix, sur le coût écologique), métro, réseaux d'ordinateur, réseaux sociaux, les liens sur les sites web.

B Matrice d'adjacence

Généralement, on utilise une matrice d'adjacence pour représenter un graphe, qu'il soit orienté ou non.

Une matrice de dimension n x p peut être vue comme un tableau de n lignes et p colonnes. Ce tableau étant complété de nombres appelés les coefficients de la matrice. On notera m_{ij} le coefficient de la ligne i et de la colonne j de la matrice m.

Dans le cadre des graphes nous aurons des matrices carrées c'est à dire de dimension n x n.

Dans une matrice d'adjacence A, l'élément A_{ij} indique si l'on peut aller du sommet i vers le sommet j et la valeur de l'arc si le graphe est pondéré.

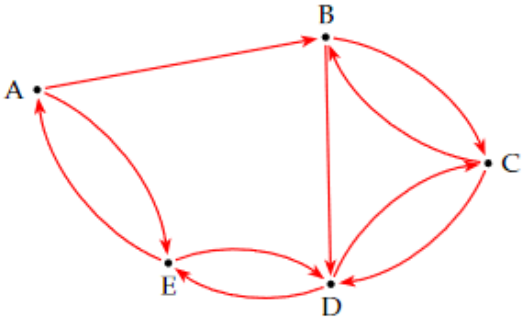
Dans le cas d'un graphe non pondéré, on placera des 0 et 1, ou tout simplement des booléens dans cette matrice.

Si le graphe est non orienté, la matrice sera symétrique : $m_{ij}=m_{ji}$.

Exemple : Une exposition est organisée dans un parc. On décide d'y instaurer un plan de circulation : certaines allées sont à sens unique, d'autres sont à double sens. Le graphe ci-contre modélise la situation.

Donner la matrice d'adjacence M de ce graphe non pondéré (dans l'ordre alphabétique) en complétant le tableau suivant :

	A	B	C	D	E
A	0	1	0	0	1
B					
C					
D					
E					



Pour la matrice d'adjacence suivante, donner le graphe correspondant :

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

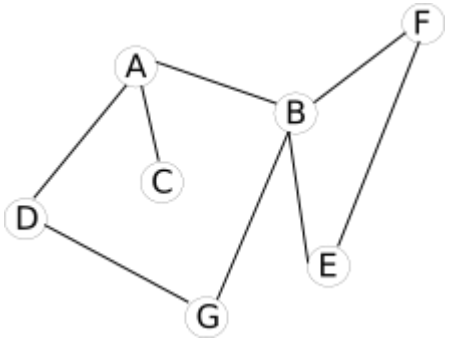
Cas d'un graphe pondéré :

Dans ce type de graphe, il suffit tout simplement de mettre dans le coefficient M(i,j) la valeur de l'arc (i,j). En revanche, il faudra faire attention puisque la valeur 0 peut être utilisée pour pondérer le graphe. On utilisera alors une valeur spéciale pour indiquer qu'il n'y a pas de chemin. Quelque fois, il s'agit d'une valeur négative si les pondérations sont toutes positives. D'une manière algorithmique, on pourrait utiliser la valeur infinie pour définir la non-existence entre deux sommets. La valeur sera donc choisie en fonction de l'application.

C Liste des successeurs et des prédécesseurs

Implémentation d'un graphe à l'aide de listes d'adjacence
Pour commencer, on définit une liste des sommets du graphe. À chaque élément de cette liste, on associe une autre liste qui contient les sommets lié à cet élément :

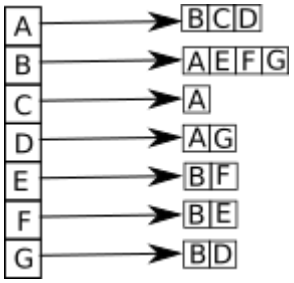
Exemple d'un graphe cartographie



La matrice d'adjacence est la suivante :

```
m = [[0, 1, 1, 1, 0, 0, 0],
     [1, 0, 0, 0, 1, 1, 1],
     [1, 0, 0, 0, 0, 0, 0],
     [1, 0, 0, 0, 0, 0, 1],
     [0, 1, 0, 0, 0, 1, 0],
     [0, 1, 0, 0, 1, 0, 0],
     [0, 1, 0, 1, 0, 0, 0]]
```

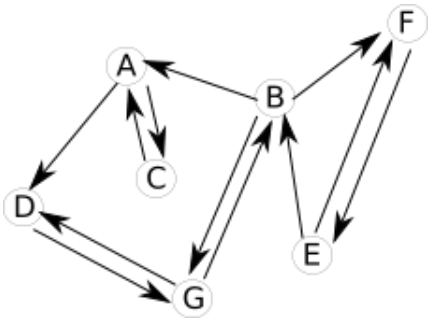
La liste d'adjacence correspondante est :

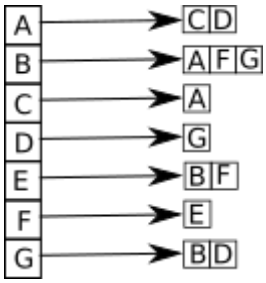
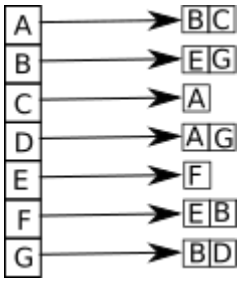


Pour les graphes orientés, il est nécessaire de définir 2 listes : la liste des successeurs et la liste des prédécesseurs. Soit un arc allant d'un sommet A vers un sommet B (flèche de A vers B). On dira que B est un successeur de A et que A est un prédécesseur de B.

La nouvelle matrice d'adjacence est la suivante :

```
m = [[0, 0, 1, 1, 0, 0, 0],
     [1, 0, 0, 0, 0, 1, 1],
     [1, 0, 0, 0, 0, 0, 0],
     [0, 0, 0, 0, 0, 0, 1],
     [0, 1, 0, 0, 0, 1, 0],
     [0, 0, 0, 0, 1, 0, 0],
     [0, 1, 0, 1, 0, 0, 0]]
```



La liste des successeurs correspond à l'extraction des lignes de la matrice d'adjacence	La liste des prédécesseurs correspond à l'extraction des colonnes de la matrice d'adjacence
	

<u>Exercices :</u>

Pour la matrice adjacence précédente, proposer deux fonctions en Python permettant d'extraire la liste des successeurs et la liste des prédécesseurs en fonction de la ligne ou de la colonne à extraire.

D Algorithmes dans les graphes

Pour pouvoir coder les algorithmes, il va falloir au préalable arriver à coder un graphe. Dans la suite, on suppose que l'on code les graphes récursivement à partir de sommets liées les uns aux autres par la liste des successeurs, prédécesseurs.

Le travail déjà fait sur les arbres va nous servir ici, pour le parcours en largeur et en profondeur, les principes seront les mêmes.

Parcours en largeur

Le parcours en largeur (BFS Breadth First Search) d'un graphe est le plus simple.

- On visite le sommet de départ.
- On observe ensuite les sommets successeurs du sommet de départ et on les enfile (on les place dans une liste les uns après les autres) sauf s'ils ont déjà été visités (c'est la différence avec l'arbre, on n'était pas obligé de retenir que l'on avait déjà visité un sommet vu qu'un seul chemin mène au sommet).
- S'ils n'ont jamais été visité, on les rajoute à la liste des visités (oui, on ne veut pas tourner en rond)
- On défile (on prend le premier que l'on a mis FIFO)
- On recommence au point 2 tant que la file n'est pas vide.

Il va falloir visiter les nœuds d'un même niveau et se souvenir qu'ensuite on va descendre voir chacun de leurs enfants qui deviendront eux même un souvenir.

Pour garder en mémoire les nœuds visités dont on ira voir plus tard les enfants, on les places dans une liste. Mais attention, il ne faut pas prendre le dernier de la liste pour la lecture mais le premier. On est donc en présence d'une file de type FIFO (first in first out).

```
fonction parcours_largeur(sommet_depart)
        observe = créer file fifo
        visite=liste_vide
        ajouter sommet_depart à visite et à observe
        tant que observe n'est pas vide
                sommet recoit le premier élément de observe
                        (pop, l'élément est supprimé de la liste)
                afficher(noeud)  #ou toute autre action
                pour tout les successeurs de noeud
                        si successeurs n'est pas dans visite:
                                ajouter successeurs à observe
                        sinon:
                                ajouter successeurs à visite

def parcours(graphe):
        n = len(graphe)
        depuis = []
        depuis.append(0)
        dejavu = [False for k in range(n)]
        while (len(depuis) !=0):
```

```
        k = depuis.pop(0)
        if dejavu[k]:
            continue
        dejavu[k] = True
        for s in graphe[k]:
            depuis.append(s)
```

En pratique, faire un parcours en largeur permet de visiter les sommets les plus proches (en termes d'étapes) en premier. Imaginons le cas de stations de métros, on aurait d'abord les stations à la distance 1 arrêt du départ, puis toutes les stations à la distance 2 arrêts etc.

Parcours en profondeur (DFS Depth First Search)

Il s'agit de l'idée inverse de la précédente. Tant que l'on peut trouver un successeur non visité, on va au successeur. Lorsqu'on n'en trouve plus, on l'observe et on remonte. Il va donc falloir se souvenir des nœuds que l'on n'a pas affichés car on veut tous ses successeurs. Dès que l'on remonte à un sommet, on veut afficher ses successeurs. On est donc remonté au sommet juste précédent, donc le DERNIER dont on voulait se souvenir. On est donc en présence d'une pile de type LIFO (last in first of). En changeant la pile FIFO de l'algorithme de parcours en largeur par une pile LIFO, on obtient l'algorithme de parcours en profondeur!

```
fonction parcours_profondeur(sommet_depart)
    observe = créer pile Lifo
    visite=liste_vide
    ajouter sommet_depart à visite et à observe
    tant que observe n'est pas vide
        sommet recoit le dernier élément de observe
            (pop, l'élément est supprimé de la liste)
        afficher(noeud)  #ou toute autre action
        pour tout les successeurs de noeud
            si successeurs n'est pas dans visite:
                ajouter successeurs à observe
            sinon:
                ajouter successeurs à visite
```

```
def parcours(graphe):
    n = len(graphe)
    depuis = []
    depuis.append(0)
    dejavu = [False for k in range(n)]
    while (len(depuis) !=0):
        k = depuis.pop()
        if dejavu[k]:
            continue
        dejavu[k] = True
        for s in graphe[k]:
            depuis.append(s)
```